

LEVEL 4 — Selecting Data in Pandas

This guide teaches how to select columns, rows, and individual cells using simple Pandas examples. Each topic includes line-by-line code, output, and an easy explanation.

Common DataFrame Used in All Examples

Line-by-line:

```
import pandas as pd

data = {
    "Name": ["Amit", "Neha", "Raj", "Priya"],
    "Age": [22, 25, 21, 24],
    "City": ["Pune", "Mumbai", "Delhi", "Pune"],
    "Marks": [85, 92, 78, 88]
}

df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Age	City	Marks
0	Amit	22	Pune	85
1	Neha	25	Mumbai	92
2	Raj	21	Delhi	78
3	Priya	24	Pune	88

1. Selecting One Column

Code:

```
name = df["Name"]
print(name)
```

Output:

```
0    Amit
1    Neha
2     Raj
3   Priya
Name: Name, dtype: object
```

Easy explanation: `df["Name"]` selects only the Name column. The result is a Pandas Series.

2. Selecting Multiple Columns

Code:

```
result = df[["Name", "Marks"]]
print(result)
```

Output:

	Name	Marks
0	Amit	85
1	Neha	92
2	Raj	78
3	Priya	88

Easy explanation: Use double square brackets `[...]` when selecting more than one column.

3. Selecting Rows

Code:

```
result = df.iloc[1]
print(result)
```

Output:

```
Name      Neha
Age       25
City     Mumbai
Marks     92
Name: 1, dtype: object
```

Easy explanation: `iloc[1]` selects the second row because Pandas positions start at 0.

4. Selecting Specific Cells

Code:

```
result = df.iloc[1, 3]
print(result)
```

Output:

```
92
```

Easy explanation: `iloc[1, 3]` means row position 1 and column position 3. It returns the value 92.

5. `loc[]`

Code:

```
result = df.loc[1, "Name"]
print(result)
```

Output:

```
Neha
```

Easy explanation: `loc[]` selects using row labels and column names. Here row label 1 and column Name are used.

6. `iloc[]`

Code:

```
result = df.iloc[1, 0]
print(result)
```

Output:

```
Neha
```

Easy explanation: `iloc[]` selects using integer positions. Row 1 means the second row and column 0 means the first column.

7. Difference Between `loc` and `iloc`

Code:

```
print(df.loc[2, "City"])
print(df.iloc[2, 2])
```

Output:

```
Delhi
Delhi
```

Easy explanation: Both return Delhi here. loc uses labels/names; iloc uses integer positions.

8. Selecting Rows by Index

Code:

```
result = df.loc[0:2]
print(result)
```

Output:

	Name	Age	City	Marks
0	Amit	22	Pune	85
1	Neha	25	Mumbai	92
2	Raj	21	Delhi	78

Easy explanation: loc[0:2] selects rows whose labels are 0 through 2. With loc, the ending label is included.

9. Selecting Columns by Name

Code:

```
result = df.loc[:, "City"]
print(result)
```

Output:

```
0      Pune
1    Mumbai
2     Delhi
3      Pune
Name: City, dtype: object
```

Easy explanation: The colon means all rows, and "City" selects the City column.

10. Row + Column Selection

Code:

```
result = df.loc[1:2, ["Name", "Marks"]]
print(result)
```

Output:

	Name	Marks
1	Neha	92
2	Raj	78

Easy explanation: The first part selects rows 1 to 2. The second part selects Name and Marks.

11. Slicing Rows

Code:

```
result = df.iloc[0:3]
print(result)
```

Output:

	Name	Age	City	Marks
--	------	-----	------	-------

0	Amit	22	Pune	85
1	Neha	25	Mumbai	92
2	Raj	21	Delhi	78

Easy explanation: `iloc[0:3]` selects positions 0, 1, and 2. The ending position 3 is not included.

12. Slicing Columns

Code:

```
result = df.iloc[:, 1:4]
print(result)
```

Output:

	Age	City	Marks
0	22	Pune	85
1	25	Mumbai	92
2	21	Delhi	78
3	24	Pune	88

Easy explanation: The colon means all rows. `1:4` selects columns at positions 1, 2, and 3.

Quick Revision — loc vs iloc

```
loc = label-based selection
df.loc[1, "Name"]          -> row label 1, column Name
```

```
iloc = position-based selection
df.iloc[1, 0]              -> second row, first column
```

Important:

loc -> uses labels and column names

iloc -> uses integer positions

loc slicing -> ending label is included

iloc slicing -> ending position is excluded

Easy Interview Explanation

Pandas provides `loc[]` and `iloc[]` for selecting data. `loc[]` is label-based, so it is useful when you know the row label or column name. `iloc[]` is position-based, so it is useful when you know the row and column number. A simple way to remember it is: loc = label, iloc = integer location.